

Phase 1 Build Guide — Domiciliary Care Attendance & Feedback App

Stack: JavaScript everywhere (HTML/CSS/JS frontend + Node.js/Express backend + SQLite database) **Why this stack:** One language (JavaScript) for the whole app. No native mobile app needed — a mobile-friendly website works in any phone browser, which is faster to build and easier for carers to use (no app store install).

Realistic timeline for a total beginner: 8–12 weeks, a few hours a few evenings a week. Don't rush — each step below builds on the last.

Step 0: Install your tools (Day 1)

1. **VS Code** — your code editor. Download from code.visualstudio.com, install it.
2. **Node.js** — lets JavaScript run outside a browser (this is your backend engine).

Download the "LTS" version from nodejs.org.

- Verify it worked: open a terminal (VS Code has one built in — menu `Terminal > New Terminal`) and type:

```
node -v
npm -v
```

You should see version numbers, not an error.

3. **Git** — version control (saves your project's history). Download from git-scm.com. 4. **A free GitHub account** — github.com — this is where your code lives online and how you'll deploy later.

Create one folder for the whole project, e.g. `care-app`, and open it in VS Code (`File > Open Folder`).

Step 1: Learn just enough HTML, CSS, JS before touching the app (Week 1–2)

Don't skip this. Spend 1–2 weeks on **freeCodeCamp's "Responsive Web Design" and "JavaScript Algorithms and Basics"** modules (both free, freecodecamp.org). You specifically need to understand:

- HTML: tags, forms, buttons, inputs

- CSS: basic styling, so pages don't look broken on a phone
- JavaScript: variables, functions, `if` statements, arrays, objects, and especially `fetch()` (how a webpage talks to a server)

You don't need to master these — just get comfortable. You'll keep learning as you build.

Step 2: Build the backend "hello world" (Week 3)

This is your first real step in the app itself. The backend is the program that runs on a server, stores data, and answers requests from the browser.

1. Inside `care-app`, create a folder called `backend`.
2. In the terminal, `cd backend`, then run:

```
npm init -y
npm install express
```

3. Create a file `backend/server.js`:

javascript

```
const express = require('express');
const app = express();
const PORT = 3000;

app.get('/', (req, res) => {
  res.send('Care app backend is running!');
});

app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

4. Run it: `node server.js`
5. Open a browser to `http://localhost:3000` — you should see your message.

Milestone: you now have a working server. This is the foundation everything else sits on.

Step 3: Set up the database (Week 3-4)

Use **SQLite** for now — it's a database that's just a single file, no separate server to install, perfect for learning and for Phase 1's scale.

1. In `backend`, run:

```
npm install sqlite3
```

2. Design your Phase 1 tables. Keep it minimal: **carers**

- id, name, email, password_hash

clients

- id, name, address, phone

visits

- id, carer_id, client_id, scheduled_start, scheduled_end, actual_start (null until clock-in), actual_end (null until clock-out), start_lat, start_lng, end_lat, end_lng

feedback

- id, visit_id, rating (1-5), comment, submitted_at

3. Create `backend/db.js` to set up the database and tables:

```
javascript
```

```

const sqlite3 = require('sqlite3').verbose();
const db = new sqlite3.Database('./care.db');

db.serialize(() => {
  db.run(`CREATE TABLE IF NOT EXISTS carers (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT, email TEXT UNIQUE, password_hash TEXT
  )`);

  db.run(`CREATE TABLE IF NOT EXISTS clients (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT, address TEXT, phone TEXT
  )`);

  db.run(`CREATE TABLE IF NOT EXISTS visits (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    carer_id INTEGER, client_id INTEGER,
    scheduled_start TEXT, scheduled_end TEXT,
    actual_start TEXT, actual_end TEXT,
    start_lat REAL, start_lng REAL,
    end_lat REAL, end_lng REAL
  )`);

  db.run(`CREATE TABLE IF NOT EXISTS feedback (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    visit_id INTEGER, rating INTEGER, comment TEXT,
    submitted_at TEXT
  )`);
});

module.exports = db;

```

Milestone: running `node db.js` once creates `care.db` with all four tables inside it.

Step 4: Build the core API endpoints (Week 4-6)

These are the "doors" your frontend will knock on. Add these to `server.js` (or split into route files once you're comfortable):

- `POST /api/login` — carer logs in with email/password
- `GET /api/visits/today?carer_id=X` — get today's visits for a carer

- `POST /api/visits/:id/clock-in` — records actual_start + GPS coords
- `POST /api/visits/:id/clock-out` — records actual_end + GPS coords
- `POST /api/feedback` — client/family submits a rating + comment for a visit

Example of the clock-in endpoint:

```
javascript

app.use(express.json());

app.post('/api/visits/:id/clock-in', (req, res) => {
  const { id } = req.params;
  const { lat, lng } = req.body;
  const now = new Date().toISOString();

  db.run(
    `UPDATE visits SET actual_start = ?, start_lat = ?, start_lng = ? WHERE id = ?`,
    [now, lat, lng, id],
    function (err) {
      if (err) return res.status(500).json({ error: err.message });
      res.json({ success: true, clockedInAt: now });
    }
  );
});
```

Build and test each endpoint one at a time using a tool called **Postman** (free, postman.com) or **Thunder Client** (a VS Code extension) — this lets you send test requests without needing the frontend built yet.

Milestone: you can manually trigger a clock-in and see the database row update.

Step 5: Build the frontend pages (Week 6–8)

Create a `frontend` folder with plain HTML/CSS/JS (no framework needed yet — keep it simple).

Pages you need for Phase 1:

1. **Login page** — carer enters email/password
2. **Today's visits page** — list of scheduled visits with "Start visit" / "End visit" buttons
3. **Feedback page** — a simple public page (no login) that a client/family opens via a link sent after each visit, with a star rating and comment box

For the GPS capture on clock-in, use the browser's built-in Geolocation API:

```
javascript

navigator.geolocation.getCurrentPosition(async (position) => {
  const { latitude, longitude } = position.coords;

  const res = await fetch(`/api/visits/${visitId}/clock-in`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ lat: latitude, lng: longitude })
  });

  const data = await res.json();
  console.log('Clocked in at', data.clockedInAt);
});
```

Make sure every page is **mobile-friendly**: use a simple CSS layout, big tappable buttons (carers will be using this one-handed, sometimes in a rush, sometimes with poor lighting or gloves on).

Milestone: a carer can open the site on their phone, log in, see today's visits, and tap to clock in/out. A family member can open a link and leave feedback.

Step 6: Connect frontend to backend and test end-to-end (Week 8–9)

Run backend and frontend together locally, and walk through the full flow yourself:

1. Log in as a test carer
2. See a test visit
3. Clock in → check the database row updated
4. Clock out → check again
5. Open the feedback link → submit a rating → check it landed in the `feedback` table

Fix bugs as you find them. This step always takes longer than expected — budget real time for it.

Step 7: Deploy so it's usable outside your laptop (Week 9–10)

1. Push your code to GitHub (`git init`), (`git add .`), (`git commit -m "Phase 1"`), then push to a new GitHub repo).

2. Deploy the backend on **Render.com** (free tier is fine to start) — it connects directly to your GitHub repo and redeploys automatically when you push updates.
3. For the database, note that SQLite's single-file approach doesn't survive redeploys well on most hosts — at this point, migrate to Render's free **PostgreSQL** add-on (a few config changes, not a rebuild) or keep testing on SQLite locally and only move to Postgres before real client use.
4. Deploy the frontend as a static site (Render and Netlify both offer free static hosting).

Milestone: you have a live URL you can open on your own phone, away from your home network.

Step 8: Pilot with real users before adding anything else

Before building Phase 2 features:

- Get 1–2 real carers to use it for a few real visits
- Get 1–2 real family members to try the feedback link
- Watch them use it if you can — you'll learn more from 10 minutes of watching than from a week of guessing

Common things that come up at this stage: GPS not working indoors/poor signal, carers forgetting to clock out, family members confused by the feedback link on older phones. All fixable — but only once you see them happen.

A note on data protection before going live with real client data

Even in Phase 1, once real client names, addresses, and visit records touch your database:

- Turn on HTTPS (Render does this automatically)
 - Don't log sensitive data (like GPS coordinates or names) to your console/error logs in production
 - Password-hash carer logins — never store plain text passwords (use the `bcrypt` npm package)
 - Agree with Simple Care 4 U in writing on data ownership, retention, and your role as a data processor, before storing any real client information
-

What's genuinely optional to skip for now

- User roles/permissions beyond "carer" — add this in Phase 2
- Push notifications — Phase 2/3
- Native mobile app — the mobile website is enough for Phase 1
- Offline support — nice to have, not essential for a pilot

Good luck — build it in this order, test each milestone before moving to the next, and you'll have a working pilot in a few months of consistent part-time effort.